Catalyst Maria High School – Class Grid

Melissa Mosier
IST 659 Data Administration Concepts and Management
8/17/2022

Summary

Catalyst Maria High School is creating a database for their class grid. This is how they will assign teachers and students to their classes for the coming semester. We will not get into Human Resources, teacher and admin relationships, additional staff, materials for classes, etc.; this is not meant to be a full mapping of relationships between everyone in this system. It is only to organize class assignments.

Teachers and administration from the same department will work together to create courses for their students. They will create a number of courses for each department. Each course will have at least one section but may have more. Each section will have at least one teacher and many students.

Stakeholders: Administrators, Teachers, Students

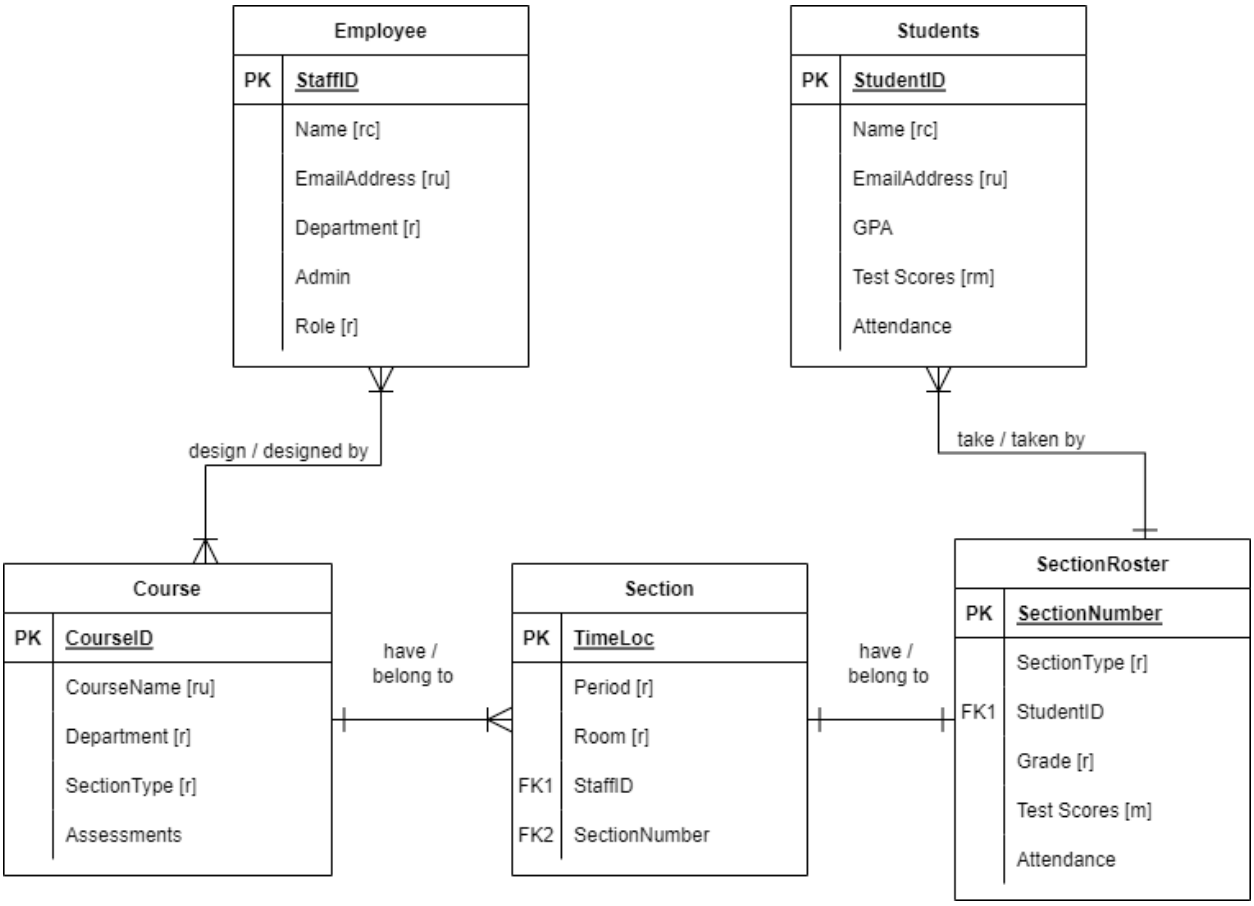
Business Rules:

- Many employees can design a course (this will usually include at least one teacher and one administrator). Many courses can be designed by an employee. Generally employees will design courses according to which department they belong.
- A course may have many sections. A section belongs to one course.
- Each section will have its own roster. A roster belongs to one section. Administrative information for each section (where it is, during which period, who teaches it) is kept separately from teacher-student information (section – DI, GENED, AP, etc.; grades; etc.)
- Many students will go on one roster (many students will take one section of a class together). A roster will only have a list of students for one class section (that class section will be taken by one roster of students).

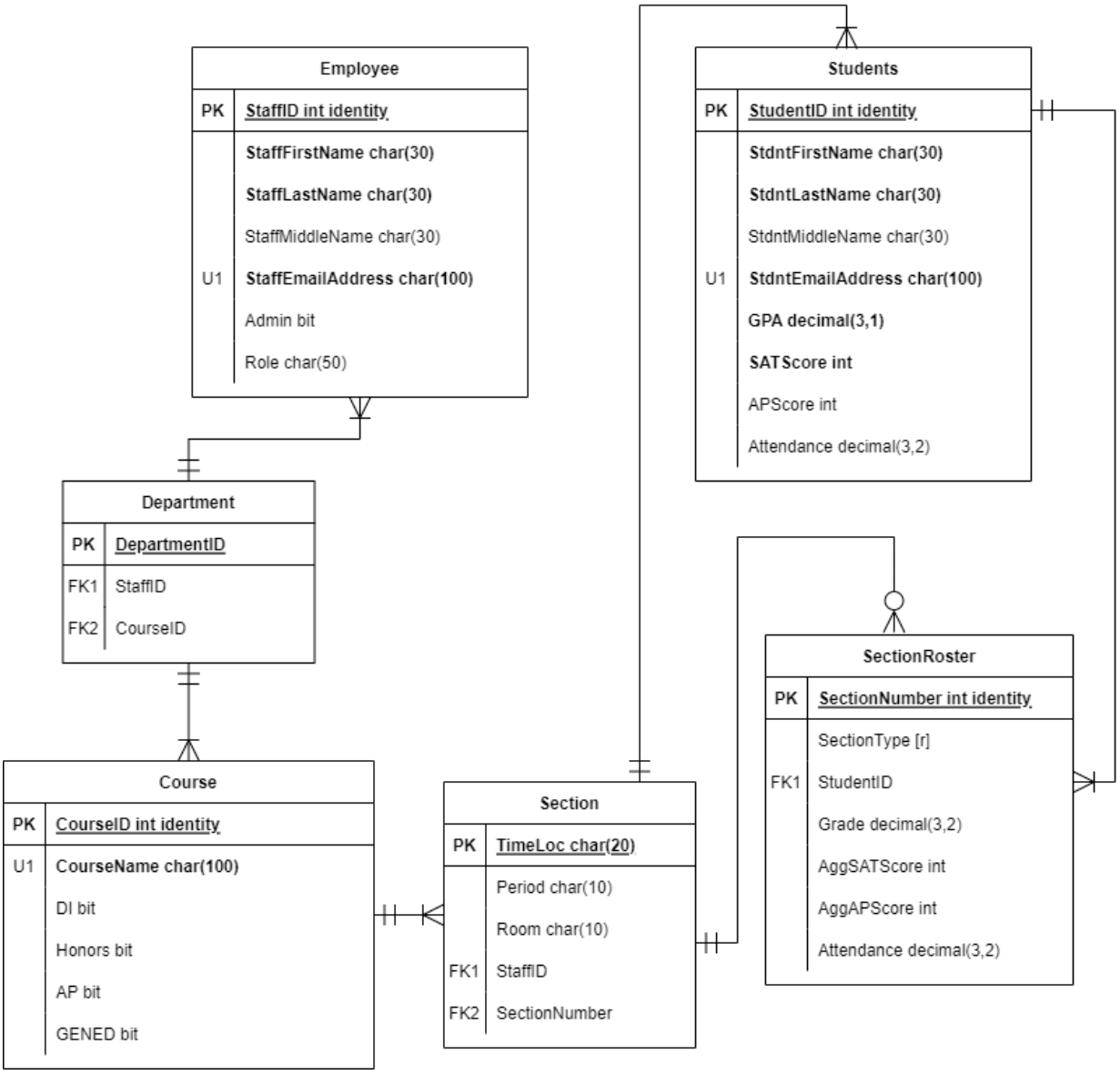
Data Questions:

- Can we pull Teacher and Student schedules on request?
- What are the metrics of students in each Course Section? (Grades, GPA, Standardized Tests, Attendance)
- Can we visualize grade trends in each Course Section? Compare between Teachers and Section Types of the same Course?

Conceptual Model



Normalized Logical Model



Additional Notes / Glossary:

Staff Table

Field Name	Data Type	Comments / Examples
StaffID	**Int identity**	**Primary Key; randomly generated**
StaffFirstName	**Char(25)**	
StaffLastName	**Char(25)**	
StaffMiddleName	Char(25)	
StaffEmailAddress	**Char(100)**	**Unique**
Admin	Bit	Ex: 0 or 1
Role	Char(50)	Ex: “Algebra 2 Teacher” vs “Math Coach” (Coach is an admin role)

Course Table

Field Name	Data Type	Comments / Examples
CourseID	**Int identity**	**Primary Key; randomly generated**
CourseName	**Char(100)**	**Unique. Ex: "Psychology"**
DI	Bit	DI = Direct Instruction
Honors	Bit	
AP	Bit	AP = Advanced Placement
GENED	Bit	GENED = General Education

Notes:

- A "1" for DI / Honors / AP / GENED indicates whether we are required to offer this type of section for this course. Later, we will indicate if a particular section is DI / Honors / AP / GENED as SectionType.

Student Table

Field Name	Data Type	Comments / Examples
StudentID	**Int identity**	**Primary Key; randomly generated**
StdntFirstName	**Char(25)**	
StdntLastName	**Char(25)**	
StdntMiddleName	Char(25)	
StdntEmailAddress	**Char(100)**	
GPA	**Decimal (3,1)**	**Ex: "3.24"**
SATScore	**Int**	
APScore	Int	
Attendance	Decimal (3,2)	This is a percentage stored as a decimal. Ex: "98.2"

Roster Table

Field Name	Data Type	Comments / Examples
SectionNumber	**Int identity**	**Primary Key; randomly generated**
SectionType	**Char(25)**	**Categorical. Ex: "GENED"**
StudentID	Foreign Key	From Student table
Grade	Decimal (3,2)	
AggSATScore	Int	Aggregated SAT Score
AggAPScore	Int	Aggregated AP Score
Attendance	Decimal (3,2)	

- This is a massive table that could be split into many smaller tables if each SectionNumber has its own table.
    - Section Number and SectionType would repeat while each StudentID gets their own row with their class Grade. This is a master roster grid.
    - In 3NF, each SectionNumber has its own table, and each StudentID still gets their own row. This is the traditional class roster as given to teachers.

Section Table

Field Name	Data Type	Comments / Examples
TimeLoc	**Char(20)**	**Primary Key. Derived from Period and Room.**
Period	**Char(10)**	
Room	**Char(10)**	
StaffID	Foreign Key	From Employee table
SectionNumber	Foreign Key	From SectionRoster table

Notes:

- No two classes can be taught in the same room during the same period. This PK (TimeLoc) could be the same as the SectionNumber PK, but this information is administration-facing, whereas SectionRoster is student-teacher-facing.

Screenshots

Staff Table

	cmhs_StaffID	StaffFirstName	StaffLastName	StaffMiddleName	StaffEmailAddress	Admin	Role
1	1	Dwayna	Sanchez	NULL	dsanchez@catdomain.xyz	1	Principal
2	2	Arnold	Twist	Patrick	aptwist@catdomain.xyz	1	DirectorSPED
3	3	Marina	Moore	Sue	msmoore@catdomain.xyz	0	Teacher
4	4	Jorge	Rudy	Cross	jcrudy@catdomain.xyz	0	Teacher
5	5	Terrell	Jones	Charles	tcjones@catdomain.xyz	0	TeacherSPED

Course Table

	cmhs_CourseID	CourseName	DI	Honors	AP	GenEd
1	1	Algebra	1	1	0	1
2	2	Biology	1	1	1	1
3	3	BasketWeaving	0	0	0	1
4	4	English	1	0	0	1
5	5	Art	0	0	0	1

Student Table

	cmhs_StudentID	StdntFirstName	StdntLastName	StdntMiddleName	StdntEmailAddress	GPA	SATScore	APScore	Attendance
1	1	Julia	Torres	Lee	juliaitorres@catdomain.xyz	2.47	1500	NULL	89
2	2	Samuel	Townsend	NULL	samueltownsend@catdomain.xyz	3.45	1280	2	25
3	3	Francis	Mosier	James	francisjmosier@catdomain.xyz	2.64	1430	NULL	54
4	4	Gabriel	Childs	Chris	gabrielcchilds@catdomain.xyz	3.98	1100	4	78
5	5	Noel	Robb	NULL	noelrobb@catdomain.xyz	3.50	980	3	97

Section Table

	cmhs_SectionNumber	Period	Room	StaffID	CourseNum	SectionType
1	1	1	336	3	1	H
2	2	2	336	4	1	G
3	3	3	336	5	1	DI
4	4	1	321	5	2	DI
5	5	2	321	3	2	AP
6	6	3	321	4	2	G
7	7	3	212	3	3	G
8	8	2	330	5	4	G
9	9	1	102	4	5	G

Roster Table

	rosternum	SectionNum	StudentNum	GradePercent	GradeLetter
1	1	3	3	89	B+
2	2	1	1	73	C-
3	3	1	2	34	F
4	4	2	4	54	F
5	5	2	5	78	C+
6	6	4	3	68	D+
7	7	4	5	84	B
8	8	5	2	62	D-
9	9	6	1	91	A-
10	10	6	4	80	B-
11	11	7	2	74	C
12	12	7	5	97	A+
13	13	8	1	54	F
14	14	8	3	85	B
15	15	9	4	94	A

Answering Data Questions:

- SQL Code to Pull Teacher Schedule

```
--What classes does Ms. Moore teach, in order?
SELECT Period, CourseName, SectionType, Room
FROM cmhs_Section
JOIN cmhs_Staff ON cmhs_Section.StaffID = cmhs_Staff.cmhs_StaffID
JOIN cmhs_Course ON cmhs_Section.CourseNum = cmhs_Course.cmhs_CourseID
WHERE StaffID = 3
ORDER BY Period
--Under WHERE, you can change StaffID to any number and find a teacher's schedule
```

100 %

Results | Messages

	Period	CourseName	SectionType	Room
1	1	Algebra	H	336
2	2	Biology	AP	321
3	3	BasketWeaving	G	212

- Pull Student Schedule

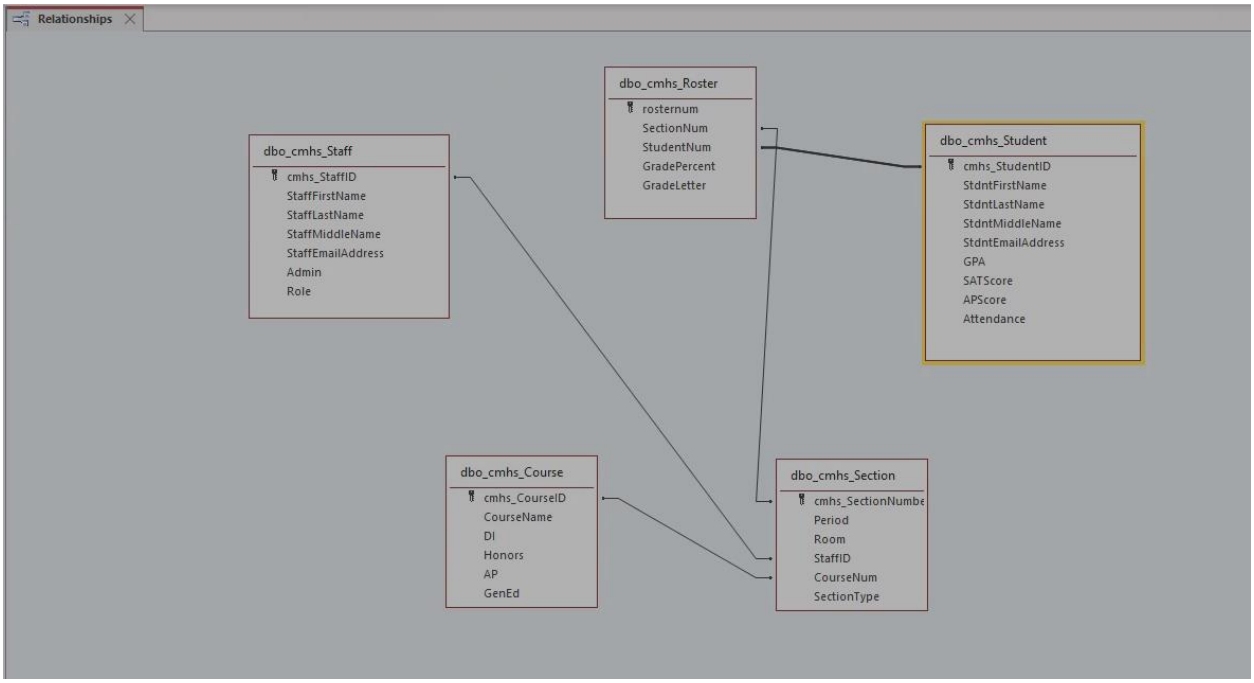
```
198
199  --Find Student Schedule
200  CREATE VIEW StudentSchedule AS
201  SELECT TOP 100 Percent StudentNum, Period, Room
202  FROM cmhs_Roster
203  JOIN cmhs_Section ON cmhs_Section.cmhs_SectionNumber = cmhs_Roster.SectionNum
204  ORDER BY Period
205
206  --How to use this VIEW
207  SELECT * FROM studentSchedule
208  WHERE StudentNum = 3
209
210
211
```

100 %

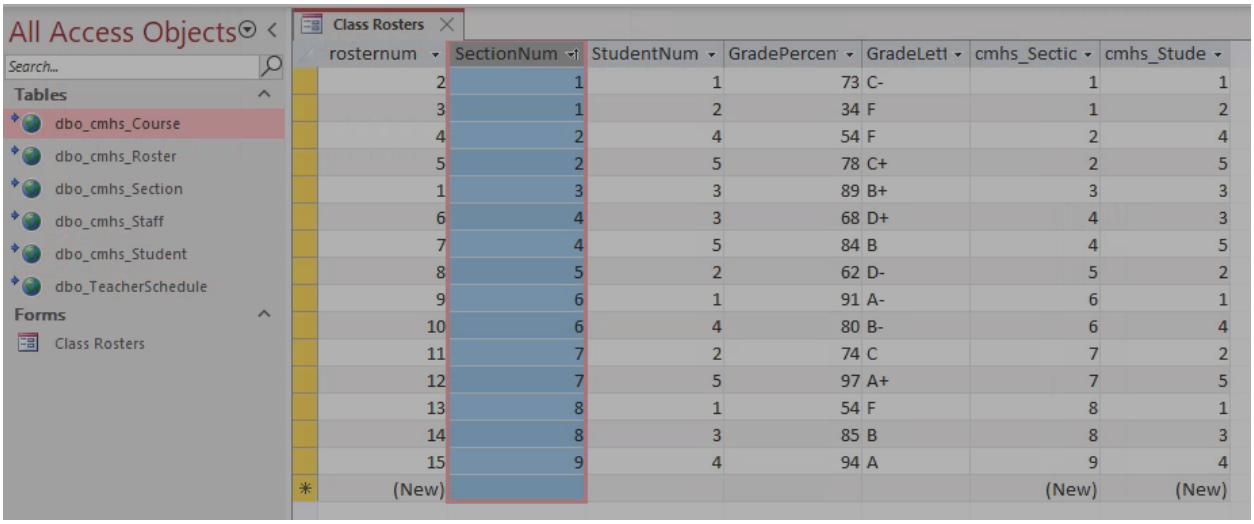
Results | Messages

	StudentNum	Period	Room
1	3	1	321
2	3	2	330
3	3	3	336

Implementation on Access

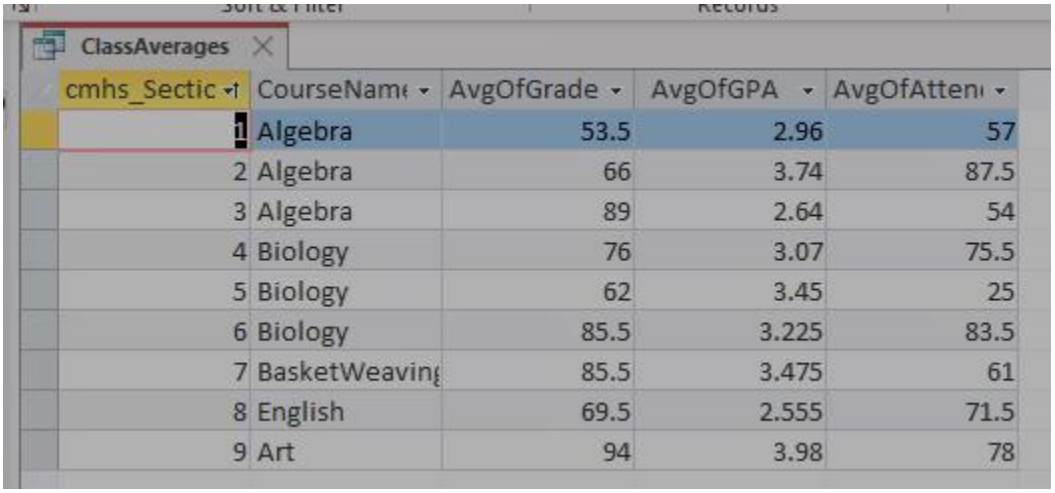


Sorting in tables



rosternum	SectionNum	StudentNum	GradePercen	GradeLetl	cmhs_Sectic	cmhs_Stude
2	1	1	73	C-	1	1
3	1	2	34	F	1	2
4	2	4	54	F	2	4
5	2	5	78	C+	2	5
1	3	3	89	B+	3	3
6	4	3	68	D+	4	3
7	4	5	84	B	4	5
8	5	2	62	D-	5	2
9	6	1	91	A-	6	1
10	6	4	80	B-	6	4
11	7	2	74	C	7	2
12	7	5	97	A+	7	5
13	8	1	54	F	8	1
14	8	3	85	B	8	3
15	9	4	94	A	9	4
(New)					(New)	(New)

- Aggregating metrics for each Course Section (Grades, GPA, Standardized Tests, Attendance)



ClassAverages

cmhs_Sectic	CourseNam	AvgOfGrade	AvgOfGPA	AvgOfAtten
1	Algebra	53.5	2.96	57
2	Algebra	66	3.74	87.5
3	Algebra	89	2.64	54
4	Biology	76	3.07	75.5
5	Biology	62	3.45	25
6	Biology	85.5	3.225	83.5
7	BasketWeavin	85.5	3.475	61
8	English	69.5	2.555	71.5
9	Art	94	3.98	78

We are able to answer all data questions!

Conclusion

I did not anticipate, when creating this database, that there would be so much reiteration in the process. At every step I found that there were modifications to be made in order for this database to function well.

- In creating the conceptual model, I conferenced with a high school Dean of Students and a database manager who uses Salesforce. They gave me feedback on content (like including SectionType, which is important for schools to retain their accreditation) and form (like figuring out how to separate and organize Sections and Section Rosters). It would turn out later that these were some the hardest parts to conceptualize for this database. It was easy to identify what information we needed to store for Staff and Students. Designing the relationships between Staff and Students, especially to their Courses, was difficult.
- In transition from the conceptual to logical models, I separated Department. It slipped my mind in transition into SQL code, since my focus fell on Student-Teacher relationships through Courses rather than their relationships to Administration. I realized it was less a structural necessity and more a referential label. I did not bother to built it into the final database, though I am sure that if a school would ask for it I were ever to implement this.
- When finally transitioning into SQL code, the Staff, Student and Course tables experienced almost no structural changes. However,
    - The Section Table needed to have SectionNumber as its Primary Key instead of that being a Key on the Roster, so that it could actually identify different classes and be referenced on the Roster when assigning students to those classes. SectionType and CourseID also needed to be in this table. I realized that this was the table that would link Courses to Staff/Teachers, whereas…

    - The Roster would link Courses to Students. AggSATScore, AggAPScore, and Attendance were all aggregated/derived attributes, so they didn't need to be included here. Rather, we only needed to gather SectionNumbers, StudentIDs, and Grades here.

- I converted Attendance and Grades to integers but upon further reflection, if I were ever to implement this with a school it would most likely need to be a percent.

This assignment does meet expectations for creating and storing a class grid (referring to Business Rules.

- Each SectionNumber is associated with a StaffID.
- Many SectionNumbers may be associated with a CourseID.
- A Section table has administrative information, like where a class is, when it takes place, who teaches it, what type of class, etc. A Roster table has Student and Grade information.
- From the Roster table, we can have both a master grid table that tells us which students take which Section of a class, and we can also pull traditional rosters for just one Section with all its students.

SQL Code

```
--Table deletions; skip for now
DROP TABLE cmhs_Section;
DROP TABLE cmhs_Staff;
DROP TABLE cmhs_Roster;
DROP TABLE cmhs_Course;
DROP TABLE cmhs_Student;



-- Creating the Staff Table
CREATE TABLE cmhs_Staff  (
  -- Columns
  cmhs_StaffID int identity,
  StaffFirstName varchar(25) NOT NULL,
  StaffLastName varchar(25) NOT NULL,
  StaffMiddleName varchar(25),
  StaffEmailAddress varchar(100) NOT NULL,
  Admin BIT,
  Role char(50),
  -- Constraints
  CONSTRAINT PK_cmhs_Staff PRIMARY KEY (cmhs_StaffID),
  CONSTRAINT U1_cmhs_Staff UNIQUE (StaffEmailAddress)
);
--Check Staff Table
SELECT * FROM cmhs_Staff;


-- Creating the Course Table
CREATE TABLE cmhs_Course  (
  -- Columns
  cmhs_CourseID int identity,
  CourseName varchar(100) NOT NULL,
  DI BIT,
  Honors BIT,
  AP BIT,
  GenEd BIT
  --Constraints
  CONSTRAINT PK_cmhs_Course PRIMARY KEY (cmhs_CourseID),
  CONSTRAINT U1_cmhs_Course UNIQUE (CourseName)
);
--Check Course Table
SELECT * FROM cmhs_Course;


-- Creating the Student Table
```

```
CREATE TABLE cmhs_Student  (
  -- Columns
  cmhs_StudentID int identity,
  StdntFirstName varchar(25) NOT NULL,
  StdntLastName varchar(25) NOT NULL,
  StdntMiddleName varchar(25),
  StdntEmailAddress char(100) NOT NULL,
  GPA DECIMAL(3,2),
  SATScore int,
  APScore int,
  Attendance int
  -- Constraints
  CONSTRAINT PK_cmhs_Student PRIMARY KEY (cmhs_StudentID),
);
--Check Student Table
SELECT * FROM cmhs_Student;


-- Creating the Course Section Table
CREATE TABLE cmhs_Section  (
  -- Columns
  cmhs_SectionNumber int identity,
  Period varchar(10) NOT NULL,
  Room varchar(10) NOT NULL,
  StaffID int,
  CourseNum int,
  SectionType varchar(25)
  -- Constraints
  CONSTRAINT PK_cmhs_Section PRIMARY KEY (cmhs_SectionNumber)
  CONSTRAINT FK1_cmhs_Section FOREIGN KEY (StaffID) REFERENCES cmhs_Staff(cmhs_StaffID),
  CONSTRAINT FK2_cmhs_Section FOREIGN KEY (CourseNum) REFERENCES
cmhs_Course(cmhs_CourseID)
)
--Check Course Section Table
SELECT * FROM cmhs_Section;


-- Creating the Roster Table
CREATE TABLE cmhs_Roster  (
  -- Columns
  rosternum int identity,
  SectionNum int,
  StudentNum int,
  GradePercent int,
  GradeLetter varchar(5)
  -- Constraints
  CONSTRAINT PK_cmhs_Roster PRIMARY KEY (rosternum),
```

```
  CONSTRAINT FK1_cmhs_Roster FOREIGN KEY (SectionNum) REFERENCES
cmhs_Section(cmhs_SectionNumber),
  CONSTRAINT FK2_cmhs_Roster FOREIGN KEY (StudentNum) REFERENCES
cmhs_Student(cmhs_StudentID)
);
--Check Roster Table
SELECT * FROM cmhs_Roster;




--Adding data to Staff Table
INSERT INTO cmhs_STAFF
  (StaffFirstName, StaffLastName, StaffMiddleName, StaffEmailAddress, Admin, Role)
VALUES
  ('Dwayna','Sanchez',NULL,'dsanchez@catdomain.xyz',1,'Principal'),
  ('Arnold','Twist','Patrick','aptwist@catdomain.xyz',1,'DirectorSPED'),
  ('Marina','Moore','Sue','msmoore@catdomain.xyz',0,'Teacher'),
  ('Jorge','Rudy','Cross','jcrudy@catdomain.xyz',0,'Teacher'),
  ('Terrell','Jones','Charles','tcjones@catdomain.xyz',0,'TeacherSPED')
--Check Staff Table
SELECT * FROM cmhs_Staff;


--Adding data to Course Table
INSERT INTO cmhs_Course
  (CourseName, DI, Honors, AP, GenEd)
VALUES
  ('Algebra',1,1,0,1),
  ('Biology',1,1,1,1),
  ('BasketWeaving',0,0,0,1),
  ('English',1,0,0,1),
  ('Art',0,0,0,1)
--Check Course Table
SELECT * FROM cmhs_Course;


--Adding data to Student Table
INSERT INTO cmhs_Student
  (StdntFirstName, StdntLastName, StdntMiddleName, StdntEmailAddress, GPA, SATScore, APScore,
Attendance)
VALUES
  ('Julia','Torres','Lee','julialtorres@catdomain.xyz',2.47,1500,NULL,89),
  ('Samuel','Townsend',NULL,'samueltownsend@catdomain.xyz',3.45,1280,2,25),
  ('Francis','Mosier','James','francisjmosier@catdomain.xyz',2.64,1430,NULL,54),
  ('Gabriel','Childs','Chris','gabrielcchilds@catdomain.xyz',3.98,1100,4,78),
  ('Noel','Robb',NULL,'noelrobb@catdomain.xyz',3.5,980,3,97)
```

```
--Check Student Table
SELECT * FROM cmhs_Student;

--Adding data to Section Table
INSERT INTO cmhs_Section
  (Period, Room, StaffID, CourseNum, SectionType)
VALUES
        ('1','336',3,1,'H'),
        ('2','336',4,1,'G'),
        ('3','336',5,1,'DI'),
        ('1','321',5,2,'DI'),
        ('2','321',3,2,'AP'),
        ('3','321',4,2,'G'),
        ('3','212',3,3,'G'),
        ('2','330',5,4,'G'),
        ('1','102',4,5,'G')
--Check Section Table
SELECT * FROM cmhs_Section;


--Adding data to Roster Table
INSERT INTO cmhs_Roster
  (SectionNum, StudentNum, GradePercent, GradeLetter)
VALUES
  (3,3,89,'B+'),
  (1,1,73,'C-'),
  (1,2,34,'F'),
  (2,4,54,'F'),
  (2,5,78,'C+'),
  (4,3,68,'D+'),
  (4,5,84,'B'),
  (5,2,62,'D-'),
  (6,1,91,'A-'),
  (6,4,80,'B-'),
  (7,2,74,'C'),
  (7,5,97,'A+'),
  (8,1,54,'F'),
  (8,3,85,'B'),
  (9,4,94,'A')
--Check Roster Table
SELECT * FROM cmhs_Roster;


--QCHECK: So what classes does Ms. Moore teach, in order?
SELECT Period, CourseName, SectionType
FROM cmhs_Section
JOIN cmhs_Staff ON cmhs_Section.StaffID = cmhs_Staff.cmhs_StaffID
JOIN cmhs_Course ON cmhs_Section.CourseNum = cmhs_Course.cmhs_CourseID
```

```
WHERE StaffID = 3
ORDER BY Period
--Here, you can change StaffID = 3 to any number and find a teacher's schedule


--Find Student Schedule
--Aggregate class data - GPA, SAT, AP, Attendance - by section#
```